# Cross-Language Design Pattern Implementations

Design Pattern Analysis in a Vue.js-Based Social Media Application

23CSE455 Design Pattern Case Study — Part 3 | Team A13

Source application: Misskey (Vue 3 frontend, NestJS backend), commit 53ac680 | 15 pattern types | 60 implementations (Java, Python, JavaScript, C++)

# 1. Objective and selected patterns

Implement one small, equivalent problem per confirmed pattern type in four languages and compare how the pattern is expressed.

| # | Pattern: Misskey instance | Source check |
|---|---|---|
| 1 | Strategy: Logging Output Strategy | correct |
| 2 | Factory: Logging Backend Factory | partial |
| 3 | Adapter: Telemetry Backend Adapter Layer | correct |
| 4 | Template Method: Misskey Chart Processing Framework | correct |
| 5 | Proxy: AnalyticsProxy | correct |
| 6 | Observer: PreferencesManager Observer | correct |
| 7 | Two-Level Cache: RedisSingleCache L1/L2 Cache | correct |
| 8 | Cache-Aside / Lazy Loading: Federated Instance Cache-Aside | correct |
| 9 | Publish-Subscribe: Global Redis Event Publication and Streaming Subscription | correct |
| 10 | Asynchronous Message Queue / Job Queue: Federation Delivery Asynchronous Job Queue | correct |
| 11 | Front Controller: Misskey API Front Controller | partial |
| 12 | Router: Nirax Frontend Router | correct |
| 13 | Service Layer: Misskey Core Application Service Layer | partial |
| 14 | Plugin Architecture / Microkernel: Misskey Frontend Plugin Architecture | correct |
| 15 | Repository: Misskey Persistence Repository Layer | partial |

"Source check" is the re-read assessment of the cited Misskey code: 11 correct, 4 partial (kept as confirmed, with stated caveats). Scope: 4 of the 15 instances are in the Vue 3 frontend, 11 in the NestJS backend. Three audit candidates (Mediator, Object Pool, Streaming Connection Factory) stay excluded as uncertain.

# 2. Equivalent problems and the four-language approach

- Each pattern models the real Misskey collaboration at small scale (in-memory stores instead of Redis, TypeORM or the browser).
- One shared fixture per pattern in Part3/test-cases/: every language reads the same cases and expected strings.
- 97 cases in total, 388 executions (15 patterns x 4 languages).
- Each language folder holds one production file, one separate test runner and a README with the run command.
- Behaviour supplied by Misskey's frameworks, and rules added by the model, are listed under "Not modelled" in each README.

# 3. Testing and measurement methodology

- **Tests:** documented command per snippet; result is "passed N/N" printed by the runner; logs kept.
- **LOC:** lizard NLOC of the production file (no blank lines, comments or tests).
- **Complexity:** largest function CCN, lizard 1.24.1 (radon cross-check on the 15 Python files).
- **Timing:** compile first; one warm-up run; five timed runs on one machine; median of the five; whole-process time, so start-up is included.
- **Environment:** OpenJDK 17.0.19 (javac 17.0.19); CPython 3.14.6; Node.js v24.15.0; g++ 16.1.1 20260625, -std=c++17.

# 4. Actual results

| Language | Median LOC (range) | Max function CC | Median process time (range) | Tests |
|---|---|---|---|---|
| Java | 37 (22-78) | 9 | 77.1-104.2 ms | 97/97 |
| Python | 29 (14-61) | 10 | 23.8-55.3 ms | 97/97 |
| JavaScript | 26 (13-56) | 9 | 43.5-51.0 ms | 97/97 |
| C++ | 43 (28-92) | 9 | 3.3-3.7 ms | 97/97 |

Per-snippet values from data/cross_language.csv, summarised over the 15 snippets of each language.

- Time differences mostly reflect runtime start-up (JVM, interpreter, native binary), not the patterns.
- Java and C++ need more lines (declarations, closing braces, includes).
- Python uses a Protocol where Java uses a nominal interface; C++ needs an integer handle to remove std::function listeners.

# 5. Limitations and conclusion

- Four patterns are only partly supported by the source: Factory (a simple factory function), Front Controller (per-endpoint routes), Service Layer (one service read in detail), Repository (the abstraction comes from TypeORM).
- Scope: 11 of 15 instances are backend (NestJS) code; the assigned topic names the Vue.js frontend.
- The warm-up run is executed by the harness but its time is not recorded; the five timed runs and medians are.
- Cyclomatic complexity for Java, JavaScript and C++ comes from one tool; radon agrees with lizard on 14 of 15 Python files.
- The Observer oldValue behaviour was reproduced by running the verbatim Misskey lines in a stub class, not inside Misskey's frontend.
- Verification was AI-assisted; the checks were run by the same model family that produced the artifacts.

**Conclusion:** all 60 equivalent implementations pass their shared fixtures; the comparison shows how each language expresses the same collaboration, with the caveats above.